

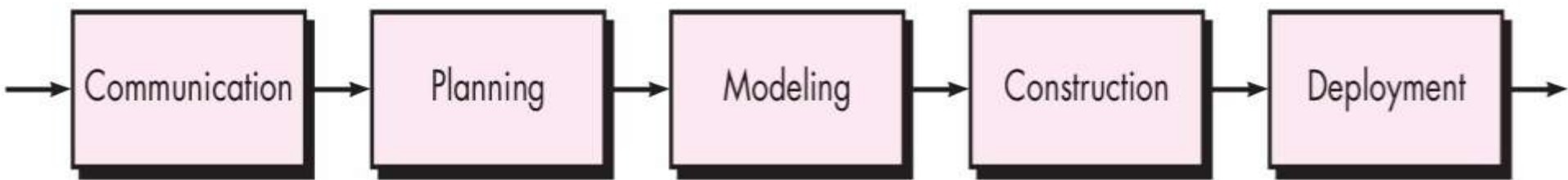
Software Engineering

Dr. Md. Nasim Adnan

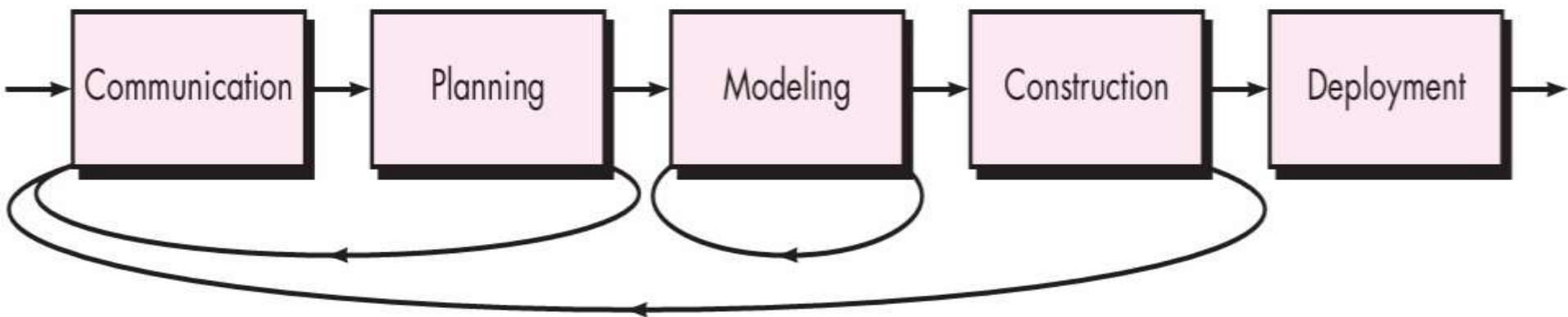
A Generic Process Model

- As I discussed in Chapter 1, a generic process framework for software engineering defines five framework activities—communication, planning, modeling, construction, and deployment.
- In addition, a set of umbrella activities—project tracking and control, risk management, quality assurance, configuration management, technical reviews, and others—are applied throughout the process.

- You should note that one important aspect of the software process has not yet been discussed.
- This aspect—called **process flow**—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time and is illustrated in the next figure.

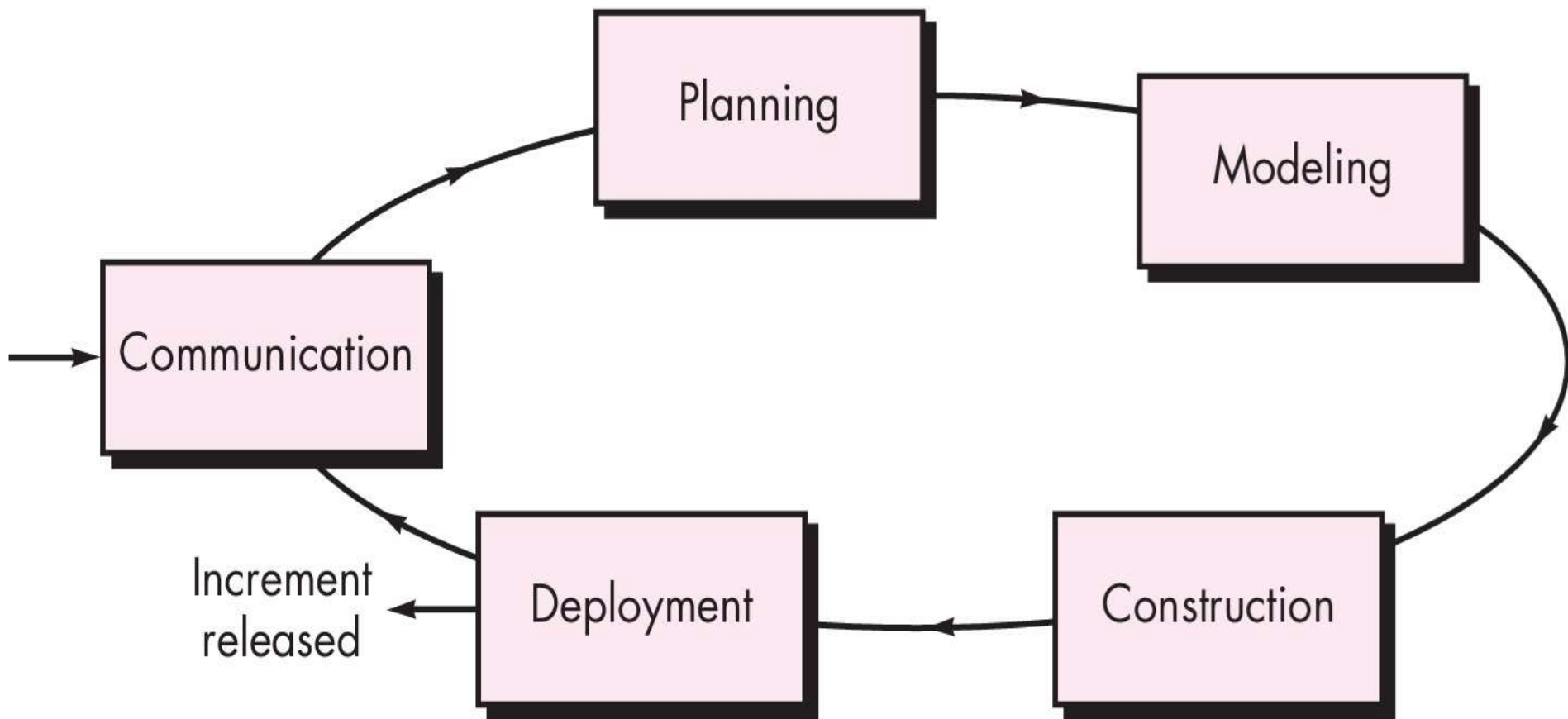


(a) Linear process flow

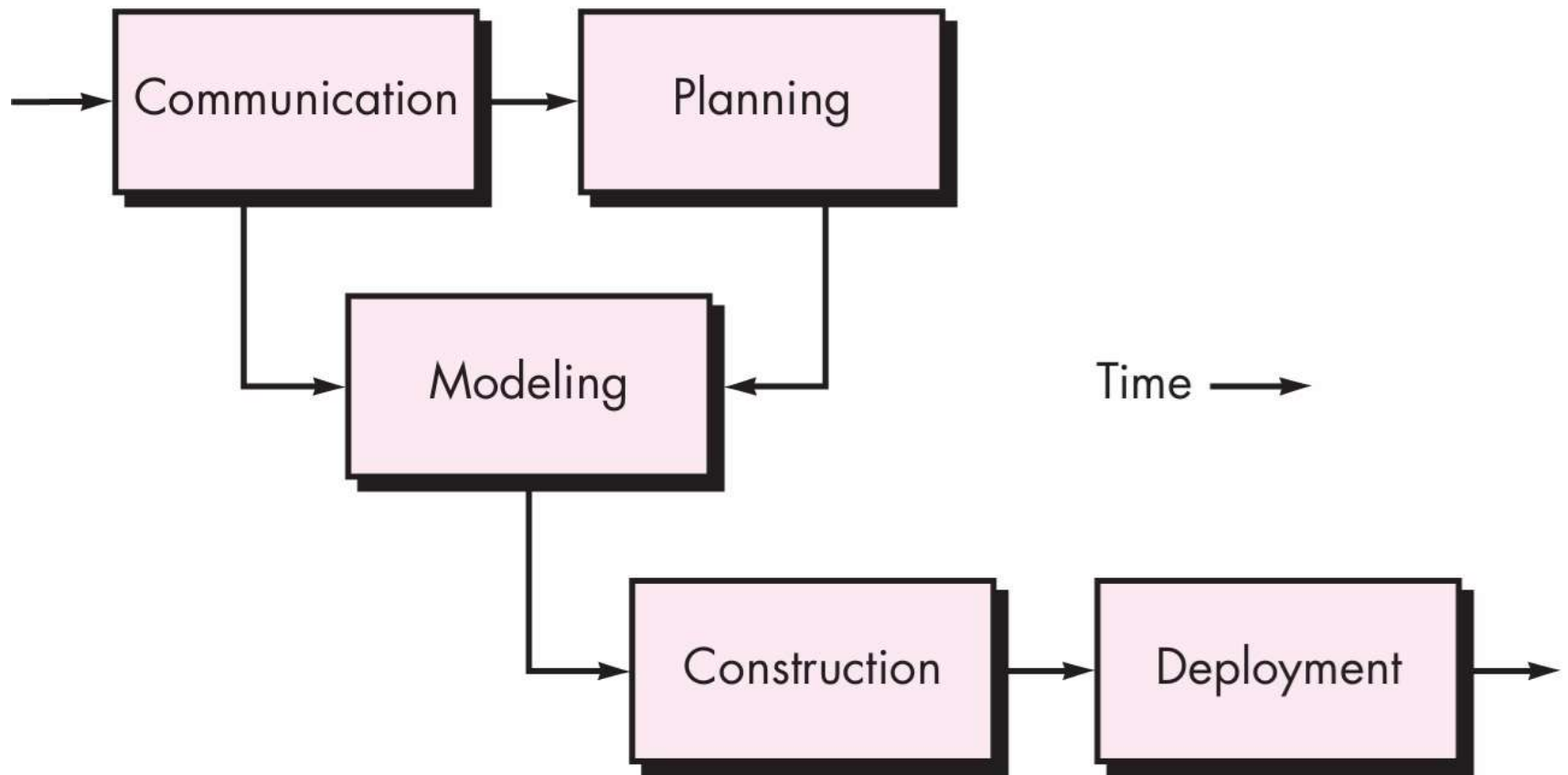


(b) Iterative process flow

- A linear process flow executes each of the five framework activities in sequence, beginning with communication and ending with deployment (Figure a).
- An iterative process flow repeats one or more of the activities before proceeding to the next (Figure b).
- An evolutionary process flow executes the activities in a “circular” manner. Each circuit through the five activities leads to a more complete version of the software (Figure c).
- A parallel process flow (Figure d) executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).



(c) Evolutionary process flow



(d) Parallel process flow

Perspective Process Models

- Prescriptive models were made to make software development more organized and less confusing.
- In the past, these models gave software teams a clear structure and a step- by- step guide (a roadmap) to follow during projects.
- But even with these models, software development can still feel messy and unpredictable — it often stays on “the edge of chaos.”

The Waterfall Model

The Waterfall Model works best when the project requirements are clear and well understood.

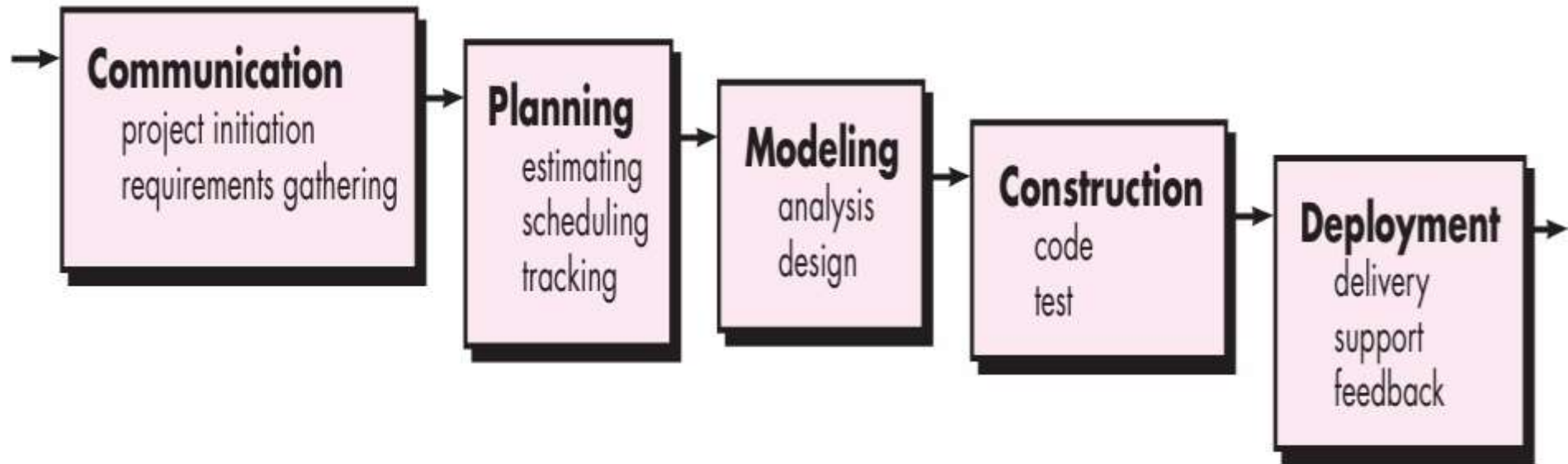
In such cases, the work moves in a straight, step-by-step order – from talking with users (communication) to giving them the final product (deployment).

This model is useful when:

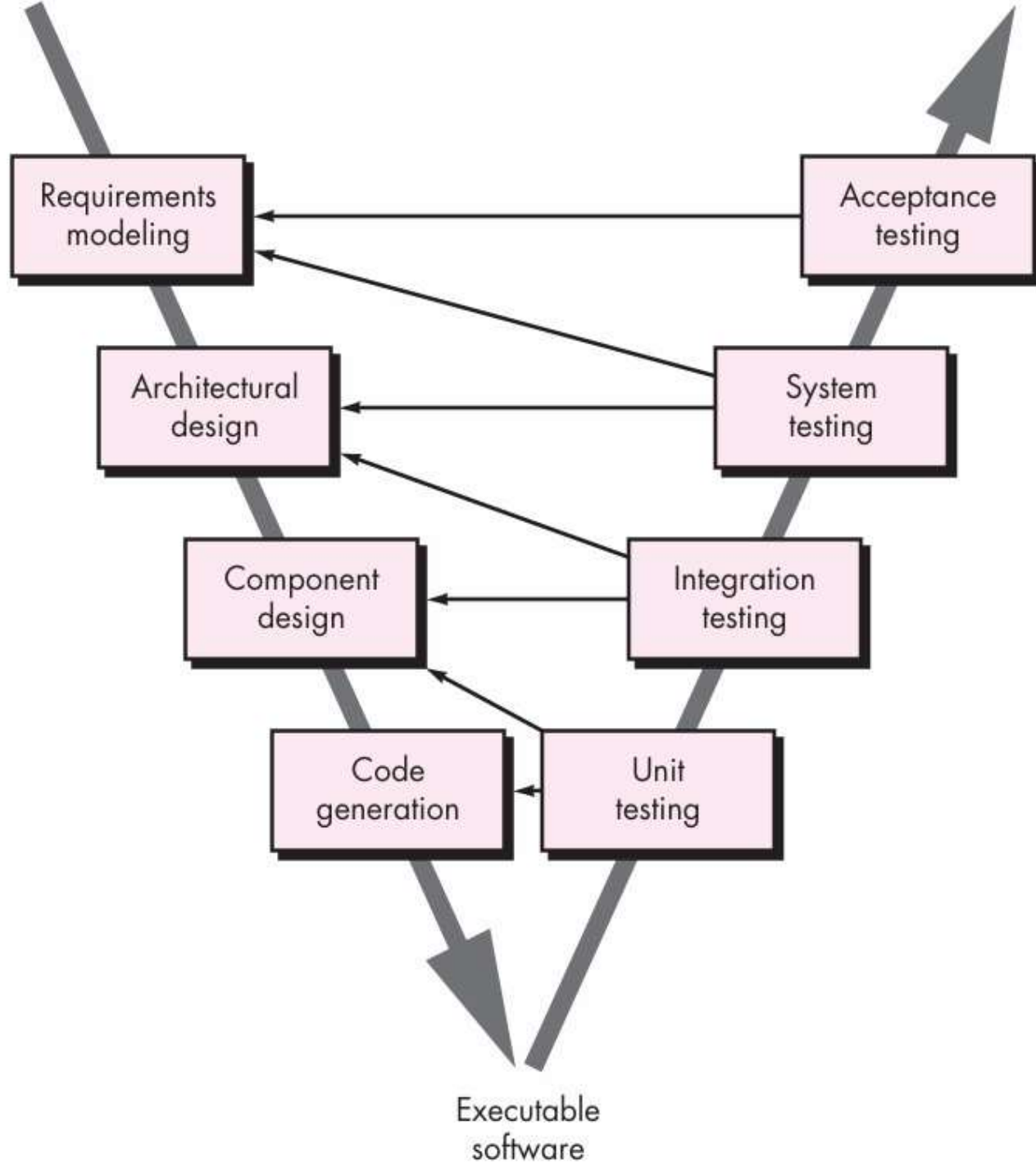
- You are making specific changes to an existing system.

- Example: Updating accounting software because of new government rules.

- In a few new projects, if the needs are clearly defined and unlikely to change.



- A variation in the representation of the waterfall model is called the V-model.
- The V-model (shown in the next figure) shows how quality assurance steps are connected to steps like planning (communication), designing (modeling), and early building (construction) of the software.



As a software team goes down the left side of the V-model, they take the basic problem and turn it into more detailed technical designs.

After writing the code, the team moves up the right side of the V, testing the system step by step to make sure each part works correctly based on the earlier designs.

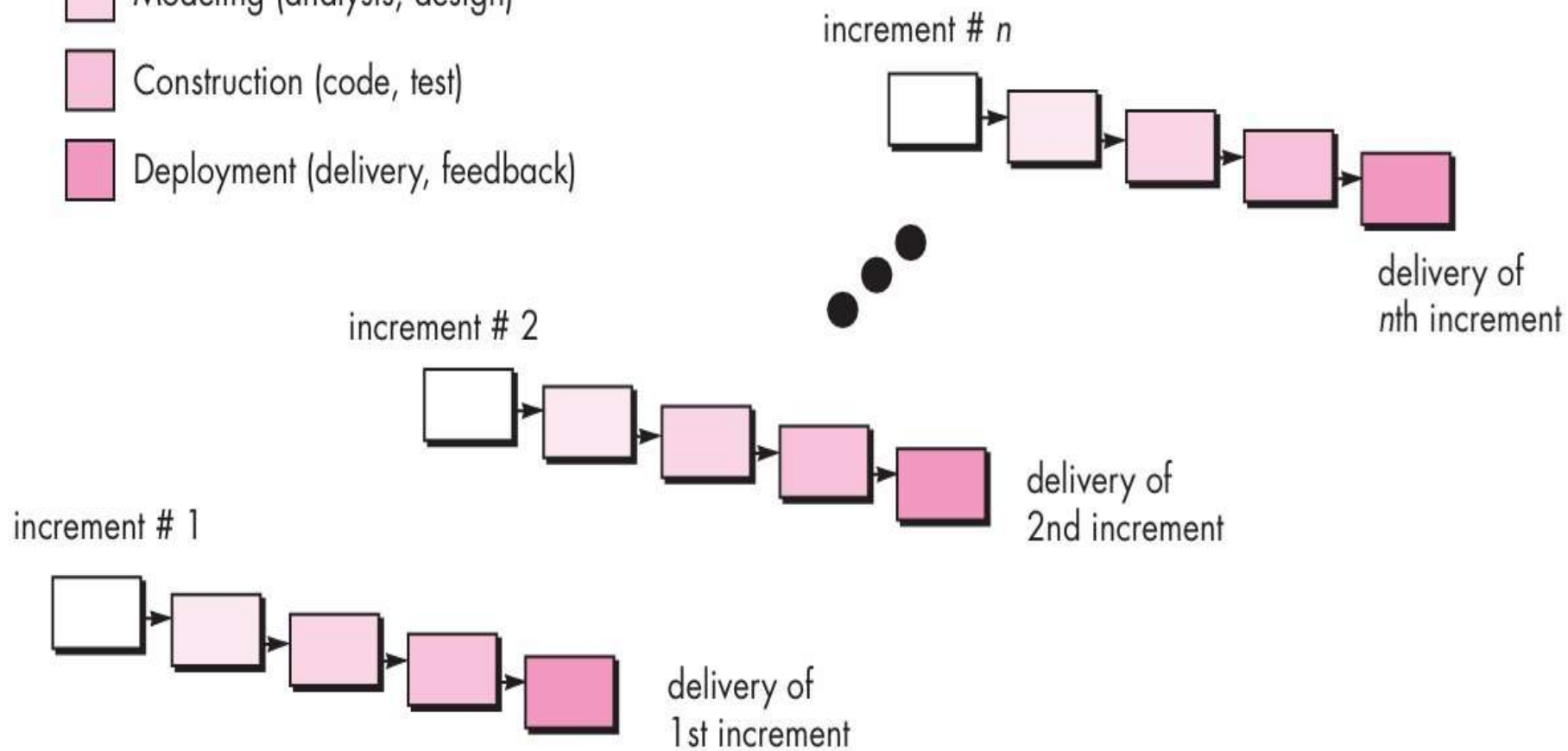
In fact, the V-model is very similar to the classic Waterfall model. It just gives a clearer picture of how testing (verification and validation) is connected to each earlier step in development.

Incremental Process Models

The incremental process model is a software development approach where a limited set of software functionality is provided to users quickly , and then that functionality is refined and expanded in later releases. It is designed to produce the software in increments. The model applies linear sequences independently as time progresses, and each sequence produces deliverable “increments” of the software.

Software Functionality and Features

- Communication
- Planning
- Modeling (analysis, design)
- Construction (code, test)
- Deployment (delivery, feedback)



Project Calendar Time

1. What is Delivered First in the Incremental Model

When the Incremental Model is used, the first thing built is usually the core product. This core product covers only the basic features needed to start using the software. Some extra features, whether already known or not yet discovered, are not included at this stage.

The customer uses the core product or reviews it carefully. Based on how the product performs and what the customer wants, a plan is made for the next version (increment). This plan includes improving the core product and adding new features.

This process is repeated again and again until the full product is finished.

2. Why the Incremental Model is Useful

With each new increment, the customer receives a working version of the software. Early versions may be simpler than the final product, but they are still useful. They help the customer test the software early and give feedback.

This model is especially helpful when there are not enough team members to build the whole software at once, or when there is a strict deadline. A small team can create the first version. If the core product is successful, more people can join later to build the next versions.

The incremental model allows faster delivery, better testing, and flexibility to adjust based on customer needs.

Evolutionary Process Models

Software, like any complex system, changes and improves over time as it is developed. As the project progresses, the business needs and product requirements often change. This makes it hard to follow a straight path to the final product.

** Business deadlines might force the team to release a limited version of the software before it's fully completed.

** Core features may be clear from the start, but extra features or details may still need to be figured out.

Because of these reasons, Evolutionary Process Models are used. These models work by improving and building upon previous versions of the software.

How Evolutionary Models Work:

**They are iterative, meaning the software is developed in cycles. Each cycle creates a more complete version of the software.

**The first version may only include basic features, and with each new version, more features and improvements are added.

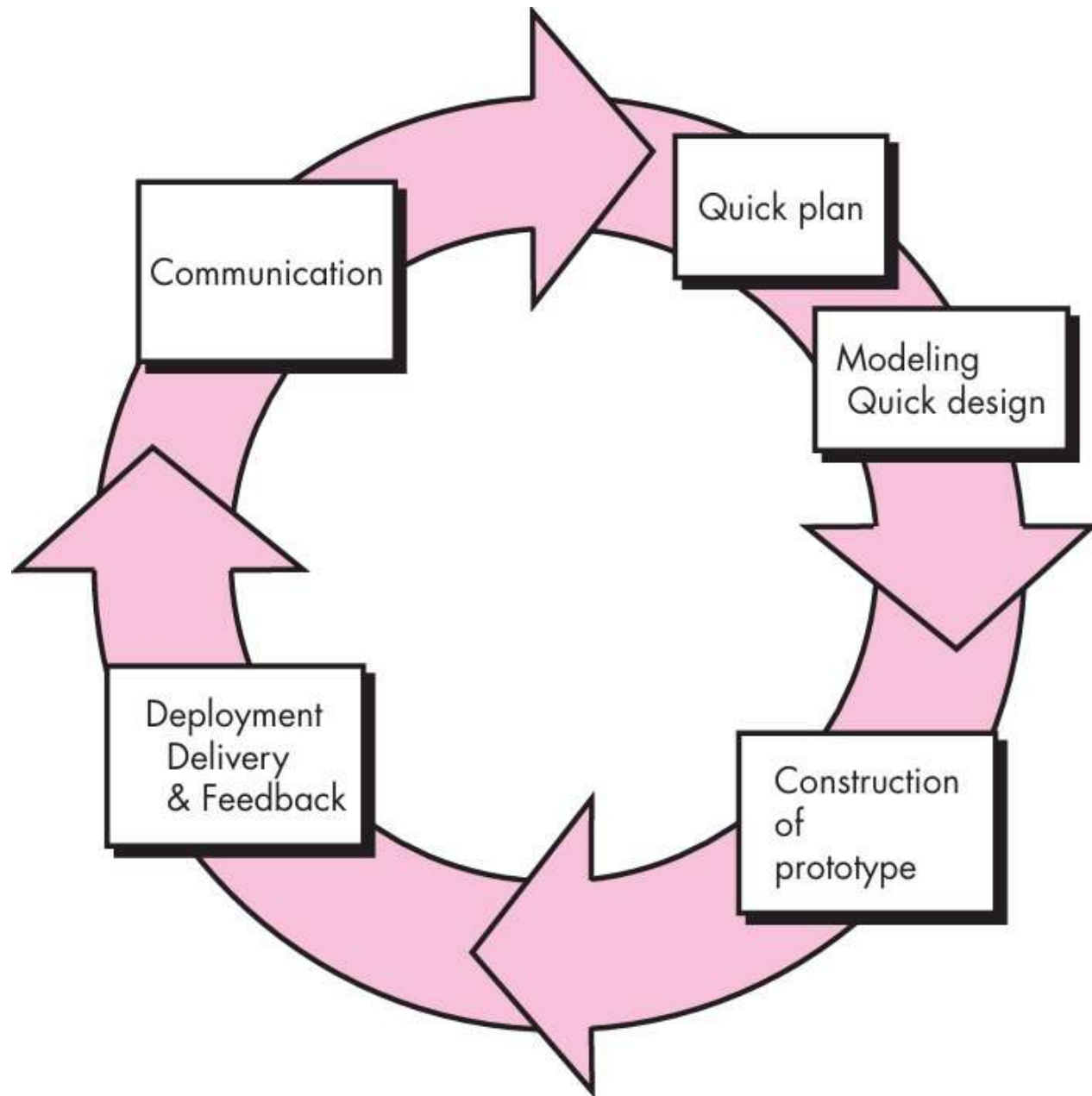
This approach helps deal with changing requirements and allows the software to improve over time until it reaches the final version.

Prototyping

Sometimes, customers know what they want in general, but they don't give clear details about all the features or functions of the software.

In such cases, the Prototyping Model is used. A prototype is a quick design of the software that is constructed, deployed, and evaluated by stakeholders. It helps both developers and customers understand the system better by building a simple version of the software early.

The prototyping paradigm begins with communication where stakeholders and developers define the overall objectives for the software, identify known requirements, and outline areas needing more detail. A prototyping iteration is planned, and a quick design is created, leading to the construction of a prototype. The prototype is then deployed and evaluated by stakeholders, whose feedback is used to refine requirements. Iteration continues as the prototype is adjusted to meet stakeholder needs and to better understand what the software must do.



The Spiral Model

The Spiral Model was originally proposed by Barry Boehm. It is an evolutionary software process model that combines the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. This model allows for the rapid development of increasingly more complete versions of software. In the spiral model, software is developed through a series of evolutionary releases. In the early iterations, the release may be a model or prototype, while in the later iterations, progressively more complete versions of the engineered system are produced.

Planning

estimation
scheduling
risk analysis

Modeling

analysis
design

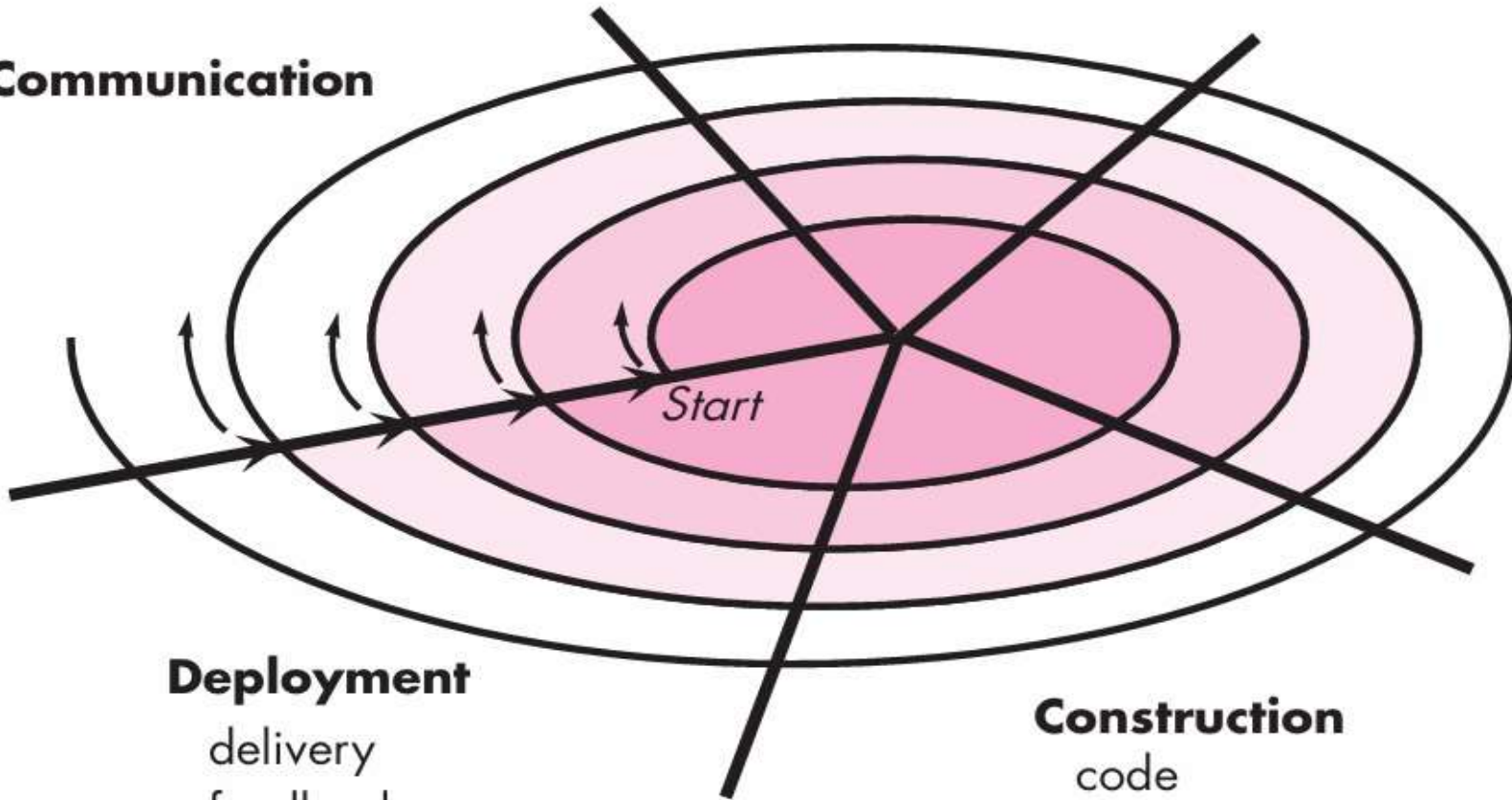
Construction

code
test

Deployment

delivery
feedback

Communication



Concurrent Models

- The concurrent development model, sometimes called concurrent engineering, allows a software team to represent iterative and concurrent elements of any of the process models.

Specialized Process Models:

Component-Based Development

The Component-Based Development (CBD) Model uses commercial off-the-shelf (COTS) software components, which are developed by vendors and offered as products. These components provide targeted functionality and have well-defined interfaces, allowing them to be integrated into the software being built. The CBD model incorporates many characteristics of the spiral model. It is evolutionary in nature, requiring an iterative approach to software development.

The Formal Methods Model

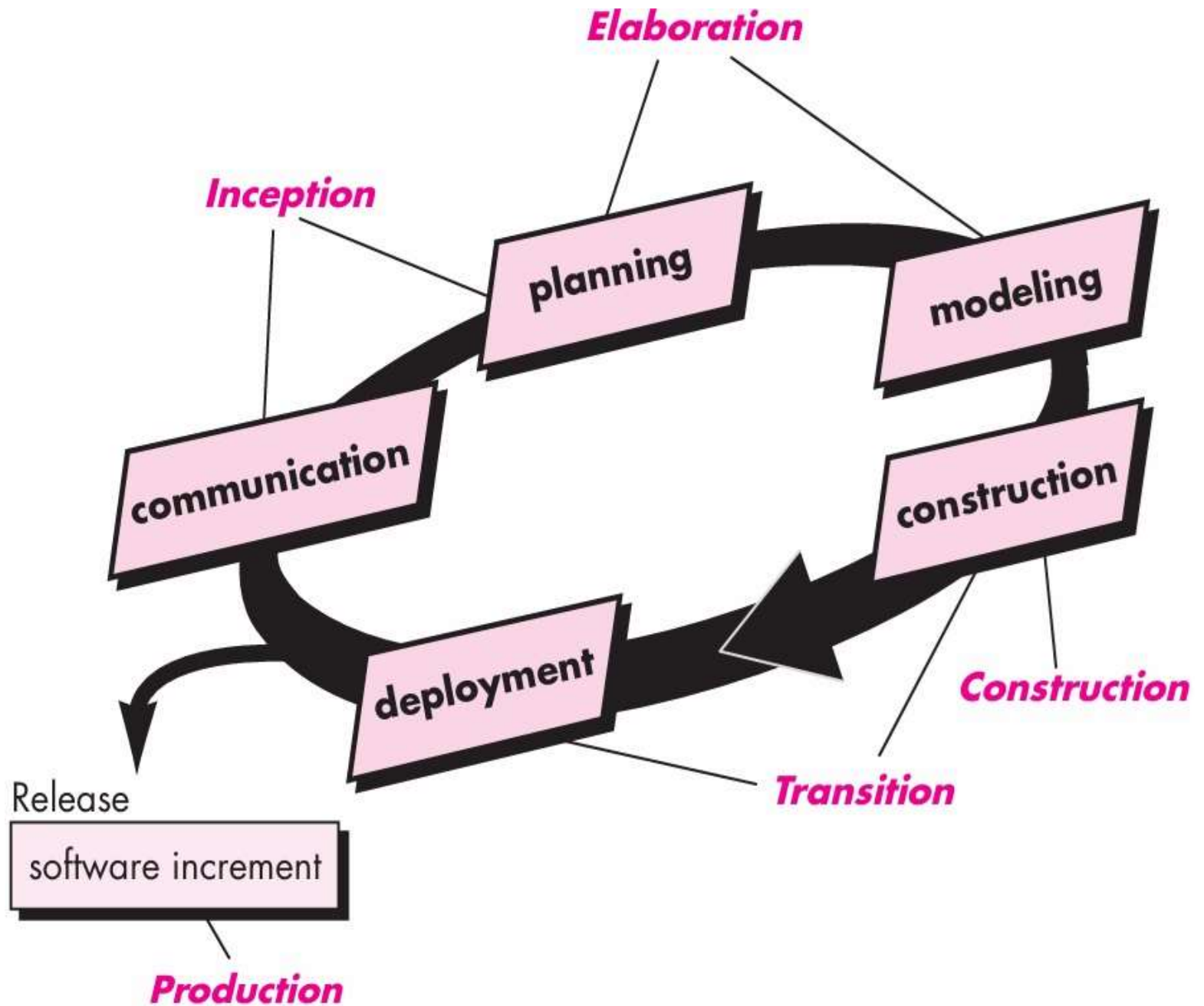
The Formal Methods Model encompasses a set of activities that lead to the formal mathematical specification of computer software. It enables the specification, development, and verification of a computer-based system by applying a rigorous mathematical notation.

Aspect-Oriented Software Development

Aspect-Oriented Software Development (AOSD), often referred to as aspect-oriented programming (AOP), is a software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects. When concerns cut across multiple system functions, features, and information, they are called crosscutting concerns. Aspectual requirements define those crosscutting concerns that impact the software architecture, such as logging, security, and error handling

The Unified Process Model

- This model uses UML—a Unified Modeling Language that contains a robust notation for the modeling and development of object-oriented systems.
- By 1997, UML became a de facto industry standard for object-oriented software development.



Personal and Team Process Models

The best software process is one that fits the people doing the work. A process designed at the organizational level works well only if it can be adapted to the project team's needs. Ideally, the process should meet both the team's and the organization's requirement

- Alternatively, the team itself can create its own process, and at the same time meet the narrower needs of individuals and the broader needs of the organization.
- It is possible to create a “personal software process” and/or a “team software process.”
- Both require hard work, training, and coordination, but both are achievable.

